Trabalho prático 2 - London Network



Cadeira: Estrutura de Dados e Algoritmos

Docentes: Maria Pinto Albuquerque

João Pedro Rolim Dias

Discentes: Martim Mourão, CDA2, Nº139570

Tomás Mourão, CDA2, Nº139617

Data de entrega: 02/06/2026

Ano letivo: 2025/2026

Índice

Índice 2
Introdução 3
Desenvolvimento 3
Análise dos Resultados, Discussão e Conclusões correspondentes às seções do enunciado 8
Análise dos resultados, Discussão e Conclusões 18
Declaração de utilização de ferramentas de Inteligência Artificial Generativa 22
Referências 22

Introdução

Este projeto tem como objetivo modelar, analisar e visualizar a rede metro de Londres recorrendo a estruturas de dados em grafo. A representação de redes de transporte público através de grafos é uma abordagem amplamente utilizada em sistemas de informação geográfica e otimização de rotas, permitindo calcular caminhos mínimos entre pontos, analisar a conectividade da rede e estudar o impacto de diferentes critérios de custo nas rotas obtidas.

Para a implementação foi utilizada a linguagem Python, com recurso às bibliotecas ***NetworkX,*** para a implementação e análise de grafos, e ***Folium***, para a visualização da rede sobre mapas geográficos reais. Os dados utilizados incluem as estações, ligações e linhas de metro de Londres, disponibilizados em ficheiros CSV.

Foram implementados dois algoritmos principais: o algoritmo de Dijkstra, para cálculo de caminhos mínimos entre estações, e o algoritmo de Kruskal, para construção da Árvore Geradora Mínima (MST) da rede. Ambos foram analisados com diferentes funções de custo, permitindo comparar os resultados obtidos e discutir o realismo e eficiência de cada abordagem.

Desenvolvimento

Fase 1: Modelação e Visualização da Rede

Para modelar a rede do Metro de Londres, utilizou-se uma abordagem baseada em Programação, recorrendo a grafos. O Tipo Abstrato de Dados (TAD) foi implementado através de uma lista de adjacências, o que se releva ideal dada a natureza esparsa da rede ferroviária urbana.

A classe ***LondonNetworkGraph*** encobre esta estrutura e processa os dados a partir de ficheiros CSV:

- Vértices (Estações): Mapeados através do ***station_id*** único, armazenando as coordenadas geográficas (latitude e longitude) e a designação oficial da estação.

- Arestas (Ligações): Representam as conexões bidirecionais entre estações adjacentes. O peso atribuído a cada aresta varia consoante a métrica analítica selecionada: custo uniforme, fixado em 1, ou distância real, calculada em quilómetros através da fórmula de Haversine.

De forma a respeitar o princípio do encapsulamento e as boas práticas de engenharia de software, estendeu-se o TAD ***Graph*** original com o método público ***iter_vertex(),*** permitindo iterar sobre os vértices sem expor os atributos privados da classe.

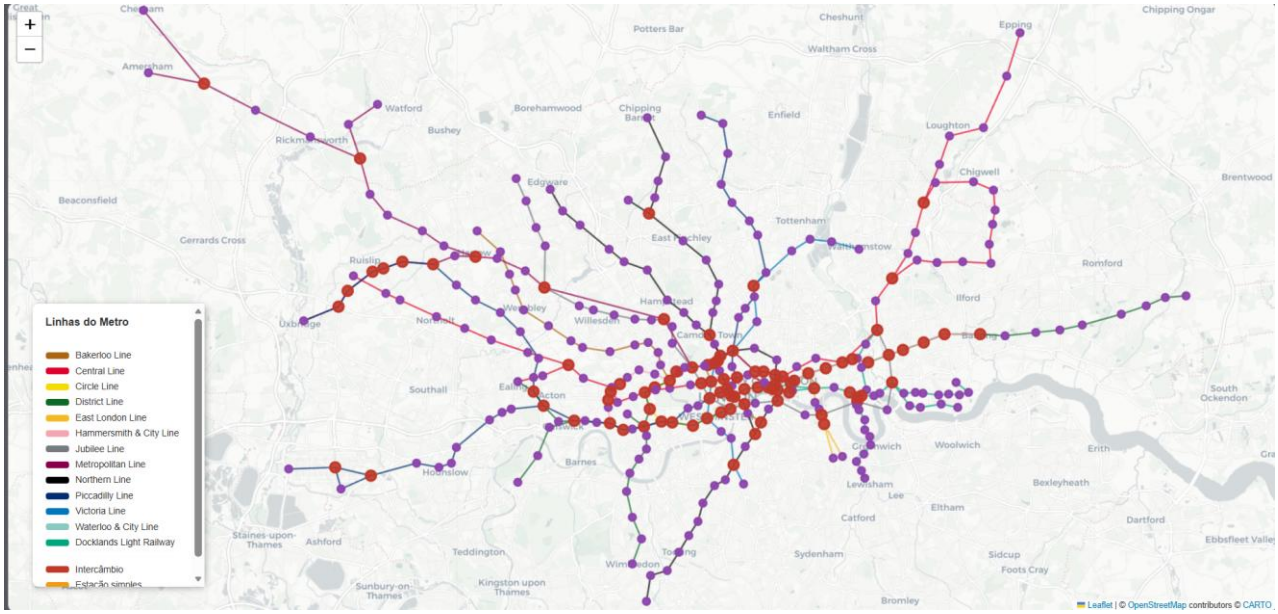


Imagem 1 – mapa completo da rede

Fase 2: Cálculo de Caminhos e Simulação

Esta fase incidiu na implementação e análise do Algoritmo de Dijkstra para a determinação do caminho mais curto entre um par de estações, origem e destino. O algoritmo foi testado em duas vertentes: através de uma implementação própria integrada do TAD ***Graph*** e recorrendo à biblioteca externa ***NetworkX***, servindo esta última como elemento de validação e aferição de desempenho.

A simulação e o comportamento do algoritmo foram avaliados sob o impacto de duas funções de custo distintas:

- **Custo uniforme:**

  - **Abordagem:** Atribui-se o peso homogéneo de 1 a todas as arestas da rede. Nesta configuração, o algoritmo abstrai-se das distâncias físicas e foca-se exclusivamente em encontrar a rota que atravesse o menor número de estações intermédias.

  - **Resultado e Impacto:** O caminho resultante otimiza o tempo puramente operacional de paragens. Contudo, do ponto de vista geográfico, a rota calculada pode incluir troços fisicamente muito longos, uma vez que a distância em quilómetros não é contabilizada na função de custo.

- **Custo por distância:**

  - **Abordagem:** O peso de cada aresta reflete a distância real em quilómetros entre estações consecutivas, calculada através da fórmula de Haversine.

  - **Resultado e Impacto:** O algoritmo prioriza trajetórias geograficamente mais diretas e curtas. Esta abordagem simula com maior fidelidade o cenário mais curto para o utilizador, embora possa, ocasionalmente, sugerir rotas com mais paragens se a soma das distâncias parciais for inferior à de uma alternativa direta mais longa.

Fase 3: Algoritmo de Kruskal

Para construir a Árvore Geradora Mínima (MST) da rede do metro de Londres, foi implementado o algoritmo de Kruskal. Ao contrário do algoritmo de Dijkstra, que determina o caminho mais curto entre dois pontos específicos, o Kruskal considera a rede na sua globalidade, selecionando iterativamente as arestas de menor custo que não formem ciclos, resultando numa árvore que liga todos os nós da rede com o menor custo total possível.

De forma a detetar e evitar ciclos, recorreu-se à estrutura de dados Union-Find (Disjoint Sets). Esta estrutura mantém conjuntos disjuntos de nós e permite verificar eficientemente se dois nós já pertencem ao mesmo conjunto — se sim, adicionar a aresta entre eles formaria um ciclo e esta é descartada. O algoritmo foi implementado ordenando primeiro todas as arestas por custo crescente, verificando para cada aresta se forma ciclo, adicionando-a à MST caso não forme, e repetindo até a MST ter n-1 arestas.

Tal como no algoritmo de Dijkstra, o algoritmo de Kruskal foi implementado com suporte a duas funções de custo: custo uniforme, em que todas as arestas têm peso 1, e distância geográfica, calculada através da fórmula de Haversine em quilómetros.

A visualização da MST foi implementada através do método **visualizar_mst()**, utilizando a biblioteca Folium. As arestas fora da MST são representadas a cinzento-claro e as da MST destacadas com a cor da respetiva linha do metro. Para a simulação, o **método simular_e_documentar_kruskal()** calcula e apresenta o número de arestas da rede original, da MST, os custos totais e a percentagem de redução de infraestrutura, gerando automaticamente um mapa HTML.

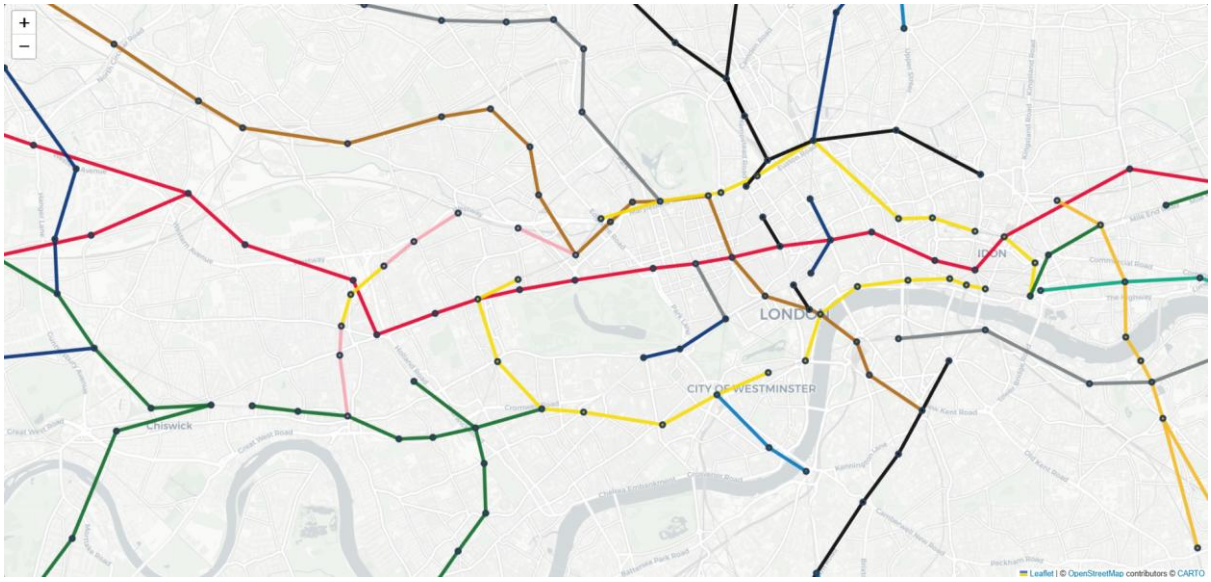


Imagem 2 – MST com custo uniforme

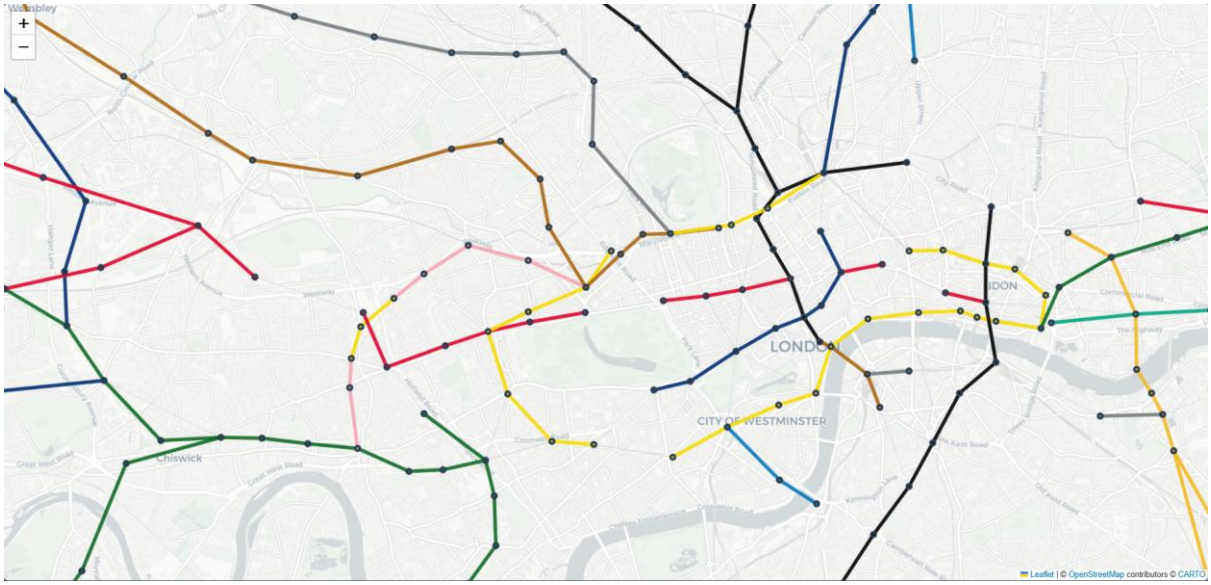


Imagem 3 – MST com custo por distância

Nestas duas imagens conseguimos ver uma das diferenças entre a linha do metro com a MST com custo uniforme e a outra MST com o custo por distância.

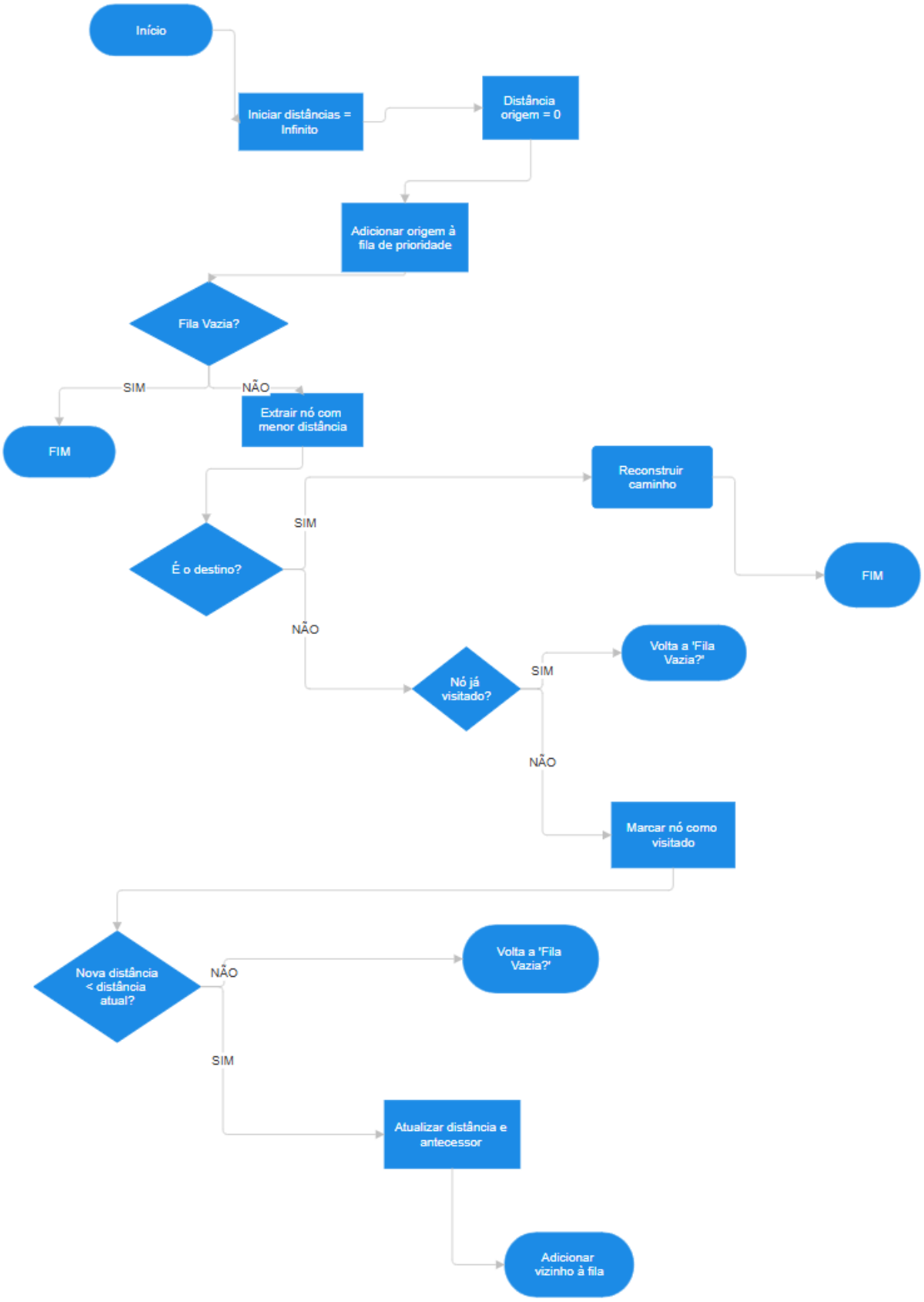


Imagem 4 – Fluxograma de Dijkstra

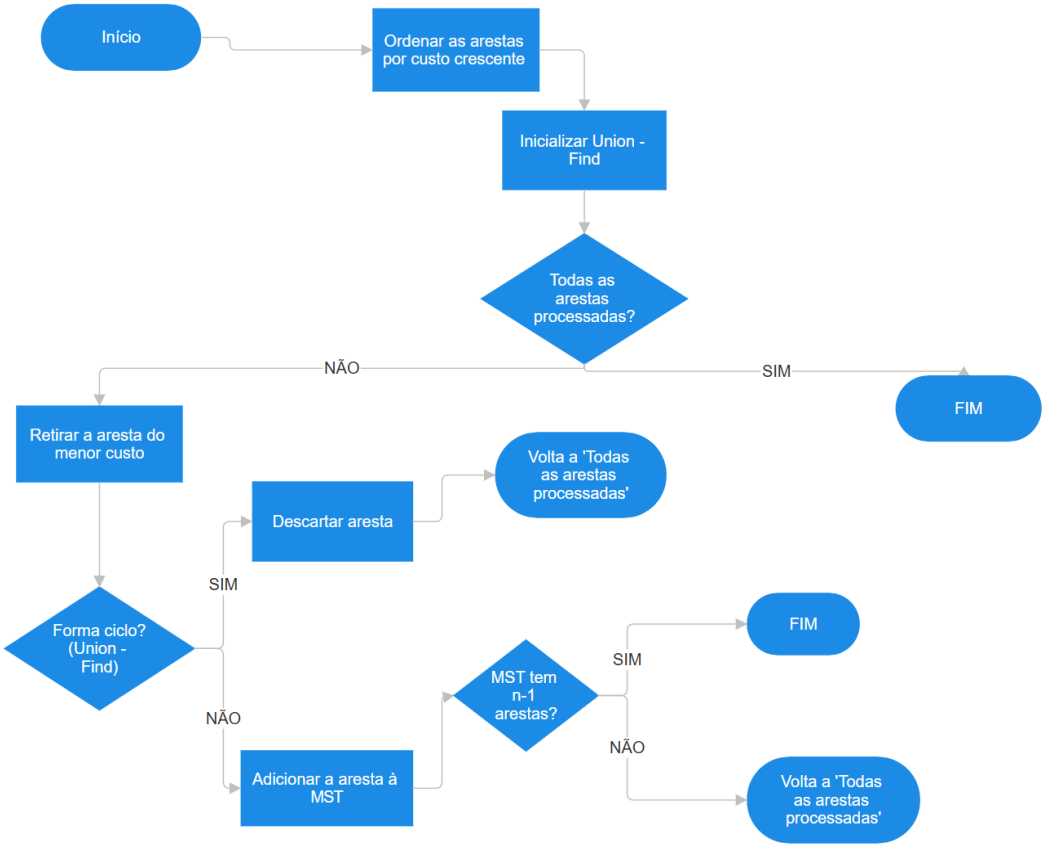


Imagem 5 – Fluxograma de Kruskal

Análise dos Resultados, Discussão e Conclusões correspondentes às seções do enunciado

Subsecção 3.1.1: Comparação das duas implementações do Algoritmo de Dijkstra

Foram implementadas duas versões do algoritmo de Dijkstra: uma utilizando a biblioteca NetworkX (‘dijkstra_nx’) e outra desenvolvida de raiz (‘dijkstra_proprio’).

Em termos de facilidade de implementação, a versão NetworkX é mais simples, requerendo apenas a construção do grafo e a chamada de funções já existentes. A implementação própria exigiu um maior esforço, sendo necessário gerir manualmente as estruturas de dados com a fila de prioridade (‘heapq’), o dicionário de distâncias e os antecessores.

Relativamente à clareza do código, a versão NetworkX é mais concisa e legível. A implementação própria, embora mais extensa, tem a vantagem de tornar explícito o funcionamento interno do algoritmo.

No que diz respeito ao tempo de execução, a versão NetworkX é geralmente mais rápida por ser bastante otimizada. A implementação própria apresenta tempo ligeiramente superiores.

Quando à flexibilidade, a implementação própria é mais flexível para adicionar novas funções de custo, como a penalização por hora de ponta, pois permite modificar diretamente o cálculo do peso de cada aresta durante a execução do algoritmo.

Subsecção 3.2.1: Explicação relativa à escolha e comportamento e uso de funções de custo no algoritmo

Foram implementadas três funções de custo no algoritmo de Dijkstra, cada uma com um comportamento distinto na determinação do caminho mínimo.

A primeira função, o custo uniforme, atribui o mesmo peso (custo = 1) a todas as arestas. Foi escolhida por ser a abordagem mais simples. Ao atribuir o mesmo peso a todas as ligações, o algoritmo foca-se apenas em minimizar a número de estações percorridas, sem ter em conta qualquer outra variável. Apesar de pouco realista, é útil como ponto de partida para comparar com as restantes abordagens.

A segunda opção, a distância geográfica, calcula o custo de cada aresta com base na distância real em quilómetros entre duas estações, recorrendo à fórmula de Haversine. Foi escolhida por ser mais realista, uma vez que reflete as distâncias efetivas da rede. Com esta função, o algoritmo tende a encontrar trajetos geograficamente mais curtos, podendo percorrer mais estações se isso implicar menor distância total.

A terceira função, a penalização por hora de ponta, multiplica a distância geográfica por um fator dependendo da hora do dia. Nas horas de ponta (7h – 9h e 17h – 19h) o custo das ligações é duplicado, enquanto nas restantes horas o comportamento é idêntico ao da função de distância geográfica. Esta função foi escolhida para simular o impacto do congestionamento na rede, gerando percursos alternativos em períodos de maior afluência.

Subsecção 3.4.1: Execução e documentação das simulações

Para a documentação das simulações foram realizados três tipos de trajetos, cada um executado com duas funções de custo diferentes: custo uniforme e distância geográfica.

O primeiro trajeto, de curta distância, foi realizado entre ‘Baker Street’ e ‘Bond Street’, duas estações próximas na mesma linha. Em ambas as funções de custo o caminho

obtido foi idêntico, percorrendo apenas 2 estações sem qualquer mudança de linha. O custo uniforme foi de 1 e o custo distância foi de 1,075 km.

O segundo trajeto, de longa distância, foi realizado entre ‘Heathrow Terminals 1,2 & 3’ e ‘Stratford’, estações nas extremidades opostas da rede. Com o custo uniforme, no caminho foram percorridas 26 estações com 7 mudanças de linha e custo total de 25. Com distância geográfica, o caminho percorreu 28 estações com 10 mudanças de linha e custo total de 34,75 km, optando por um percurso geograficamente mais curto, mas com mais transbordos.

O terceiro trajeto, com múltiplos transbordos foi realizado entre ‘Wimbledon’ e ‘Canary Wharf’. Com custo uniforme o caminho percorreu 21 estações com 2 mudanças de linha. Com distância geográfica, o resultado foi idêntico em termos de sequência e número de transbordos, com o custo total de 21,28 km.

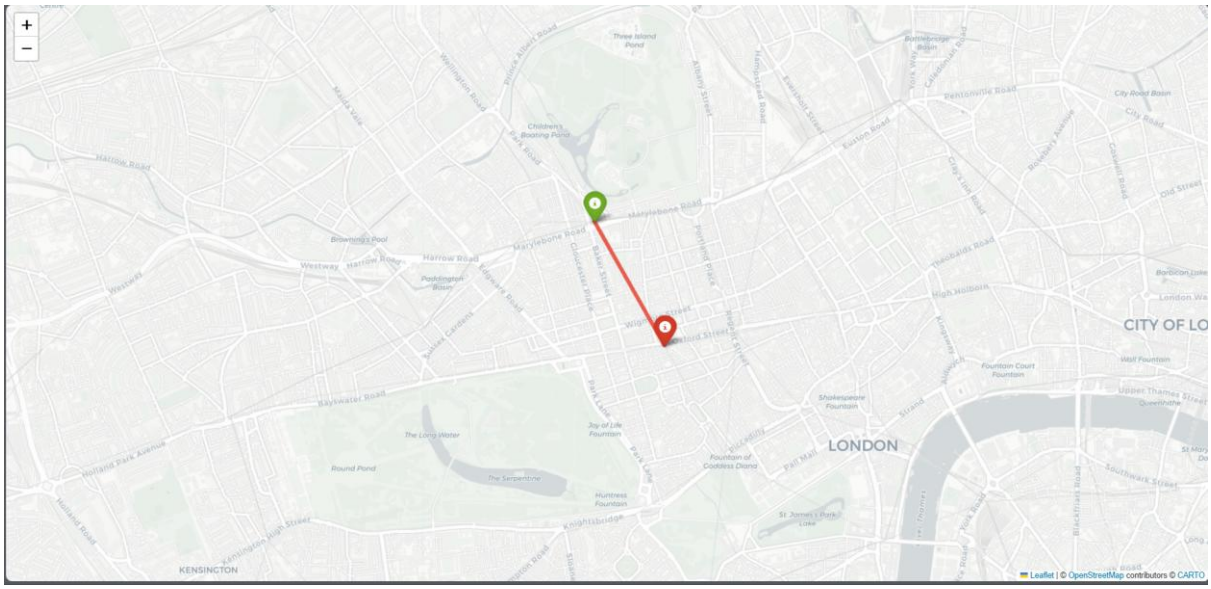


Imagem 6 – Trajeto curto (custo uniforme e por distância)

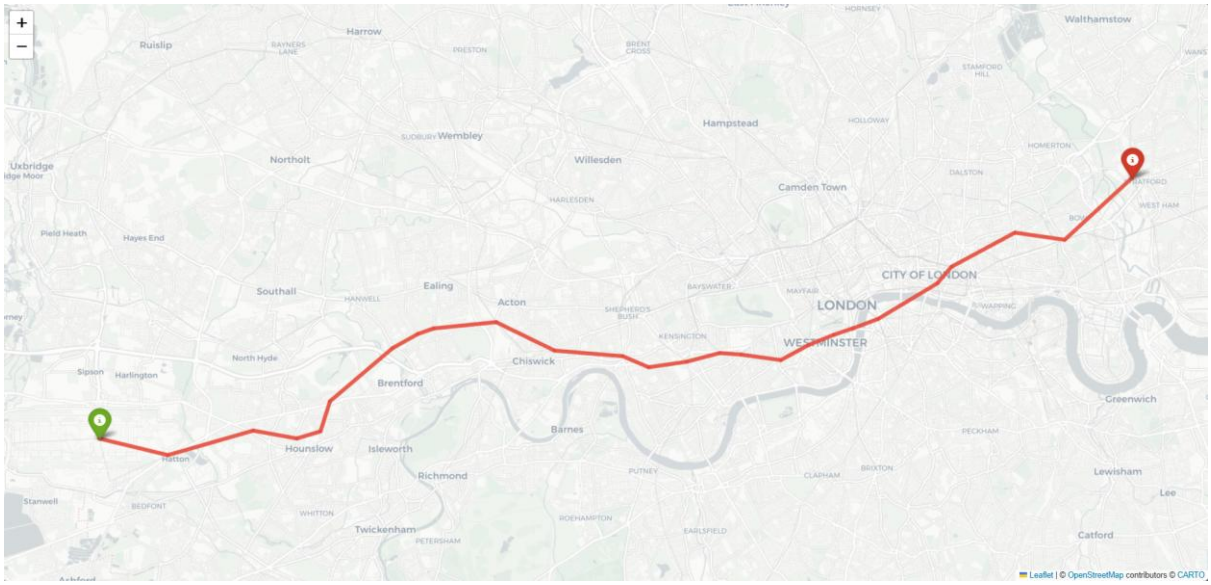


Imagem 7 – Trajeto longo uniforme

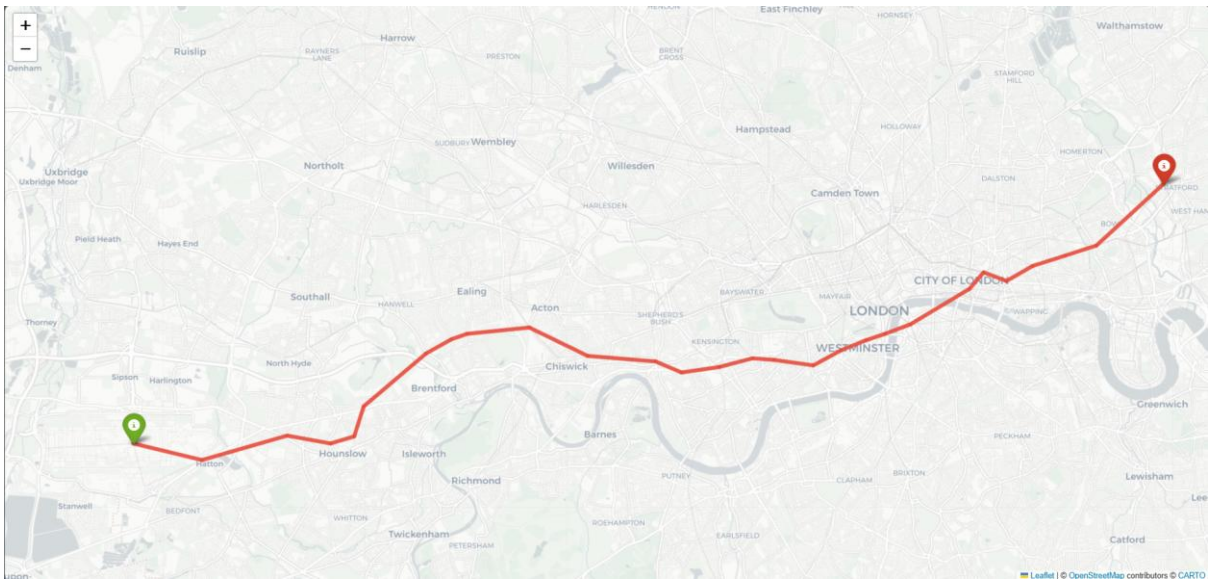


Imagem 8 – Trajeto longo distância

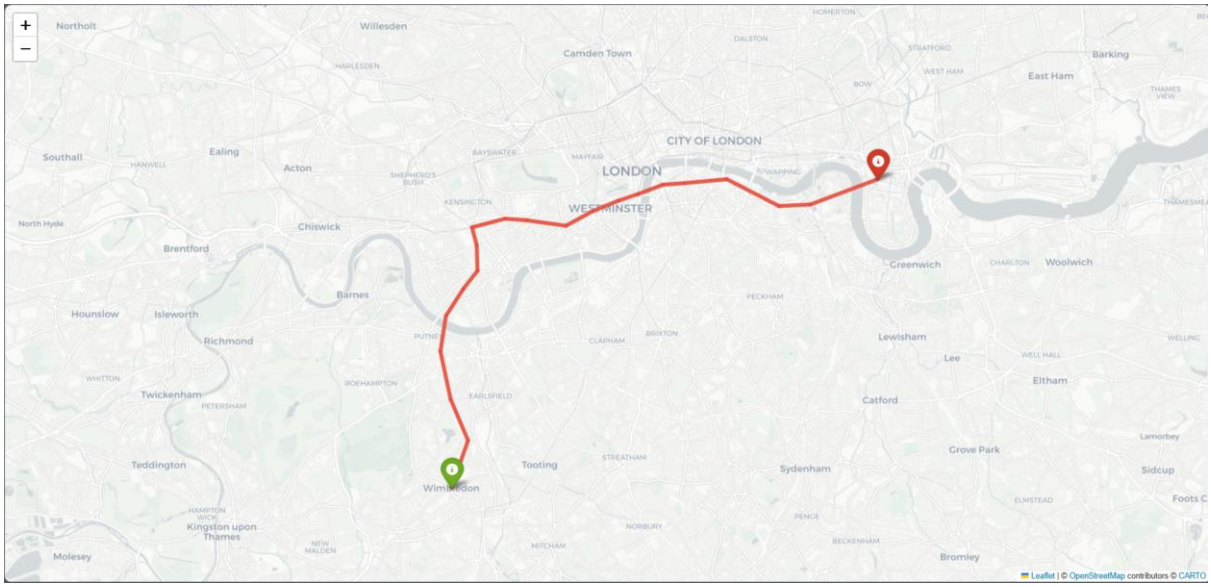


Imagem 9 - múltiplos transbordos (custo uniforme e por distância)

A tabela seguinte resume os resultados obtidos nas simulações realizadas:

Trajeto	Função de Custo	Nº Estações	Custo total	Nº Mudanças
Baker Street-> Bond Street	Uniforme	2	1	0
Baker Street-> Bond Street	Distância	2	1,075 km	0
Heathrow -> Stratford	Uniforme	26	25	7
Heathrow -> Stratford	Distância	28	34,75 km	10
Wimbledon -> Canary Wharf	Uniforme	21	20	2
Wimbledon -> Canary Wharf	Distância	21	21,28 km	2

Tabela 1 – Tabela entre os caminhos das várias simulações

Subsecção 3.4.2 - Análise comparativa dos caminhos obtidos com diferentes funções de custo e do impacto da definição das funções no resultado

A análise comparativa dos caminhos obtidos permite retirar conclusões relevantes sobre o impacto da função de custo nos resultados do algoritmo de Dijkstra

Em termos de custo total, o trajeto curto (‘Baker Street’ → ‘Bond Street’) apresentou os valores mais baixos em ambas as funções de custo, seguido do trajeto com múltiplos transbordos (‘Wimbledon’ → ‘Canary Wharf’) e por último o trajeto longo (‘Heathrow Terminals 1, 2 & 3’ → ‘Stratford’). O mesmo padrão verificou-se no número de mudanças de linha, sendo o trajeto curto o que menos transbordos exigiu, seguido do trajeto com múltiplos transbordos e do trajeto longo.

Comparando as duas funções de custo, verificou-se que no trajeto curto ambas produziram o mesmo caminho, diferindo apenas no custo total. No trajeto, longo, a distância geográfica produziu um caminho com mais estações e mais mudanças de linha, uma vez que o algoritmo privilegiou ligações geograficamente mais curtas em detrimento do número de transbordos. No trajeto com múltiplos transbordos ambas as funções produziram o mesmo caminho.

Um resultado interessante foi o facto de o trajeto longo ter apresentado mais mudanças de linha do que o trajeto designado de múltiplos transbordos, o que demonstra que a distância entre as estações de origem e de destino tem maior impacto no número de transbordos do que a escolha intencional de um trajeto com múltiplas mudanças de linha.

Em termos realísticos, a função de distância geográfica produz resultados mais próximos da realidade, uma vez que tem em conta as distâncias efetivas entre estações. No entanto, não considera outros fatores relevantes como o tempo de espera, a frequência dos comboios ou o congestionamento da rede.

Subsecção 4.2.1: Explicação relativa à escolha e comportamento do uso das funções de custo no algoritmo

A flexibilidade do algoritmo de Dijkstra reside na forma como a função de custo é definida. No âmbito deste projeto, a escolha entre duas funções de custo distintas não altera a mecânica lógica do algoritmo, mas redefine por completo o objetivo de otimização do sistema de transportes.

- **Custo uniforme:**

- Esta abordagem assume que todas as ligações entre estações adjacentes possuem um custo idêntico e homogéneo, independentemente da distância física que as separa.

- **Custo por distância:**

  - Esta métrica introduz restrições espaciais e físicas do mundo real. Utiliza a distância linear calculada através da fórmula de Haversine, baseada na latitude e longitude de cada vértice.

Subsecção 4.4.1: Execução e documentação das simulações

Para validar a robustez e a eficiência das implementações do Algoritmo de Dijkstra, foram executadas múltiplas simulações de cálculo de caminho mais curto entre pares de estações predefinidos. O processo experimental foi estruturado em dois eixos metodológicos:

- **Aferição Algorítmica (Dijkstra Próprio vs NetworkX):** Cada simulação foi executada em duplicado, primeiro recorrendo à função desenvolvida de raiz no TAD ***Graph*** (***dijkstraproprio)*** *e, posteriormente, utilizando a biblioteca de referência* ***NetworkX*** *(**dijkstra_nx).*** Esta abordagem permitiu auditar a correção lógica da nossa implementação, garantindo a correspondência exata dos caminhos e custos gerados, além de servir como base para a monitorização dos tempos de execução.

- **Parametrização do custo:** As simulações foram testadas alternando dinamicamente o parâmetro ***weight_type*** entre ***uniform*** e ***distance,*** documentando-se em consola os metadados do percurso: tempo de processamento, custo total acumulado, número de estações visitadas e número de trocas de linhas efetivas.

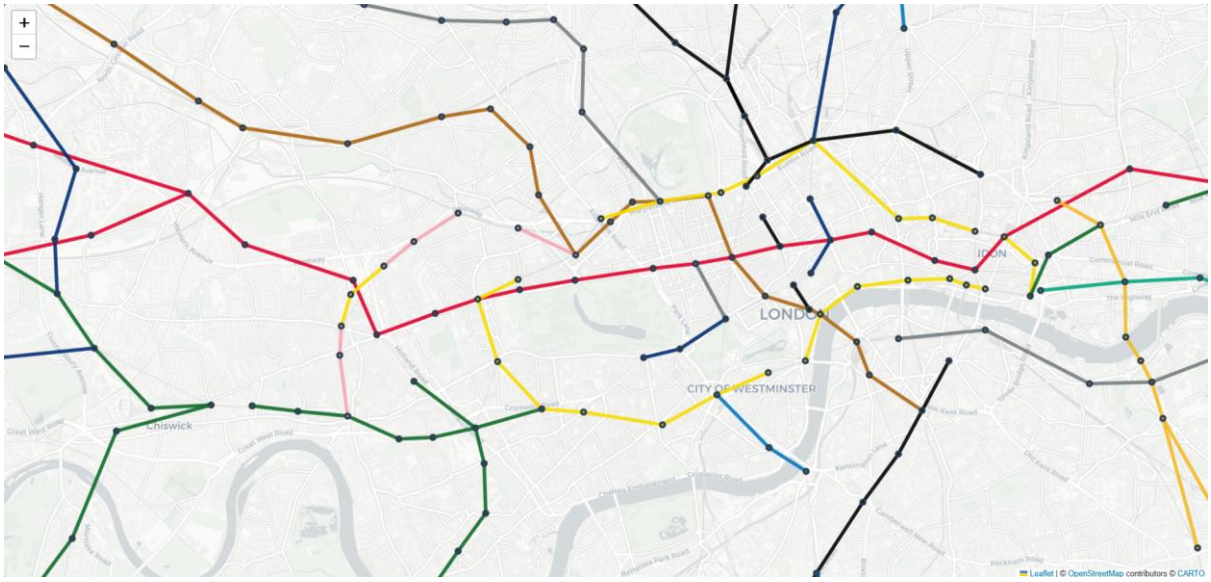


Imagem 10 – MST com custo uniforme

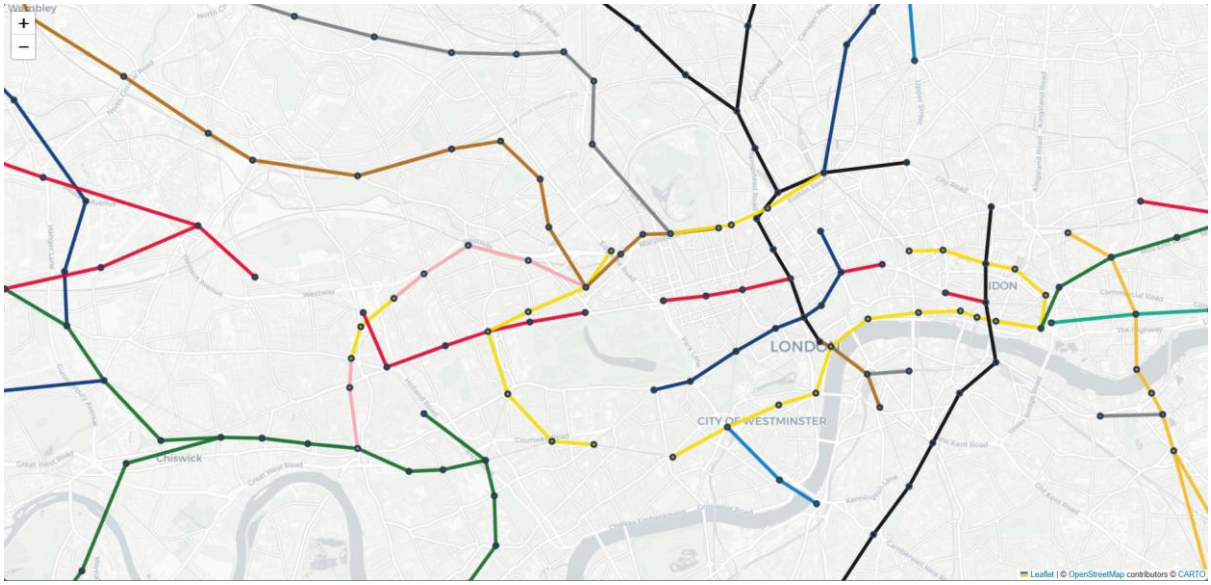


Imagem 11 – MST com custo por distância

Subsecção 4.4.2: Análise comparativa das MSTs obtidas com diferentes funções de custo e do impacto da definição das funções de resultado

A aplicação do Algoritmo de Kruskal à rede de Londres permitiu estudar a sensibilidade da topologia de uma Árvore de Custo Mínimo (MST) face à definição da sua função de custo. Sendo a MST um subgrafo que conecta todos os nós sem ciclos e com o mínimo somatória de pesos possível, a escolha da métrica altera drasticamente a infraestrutura resultante:

- **MST com custo uniforme:** Ao fixar o peso de cada aresta em 1, o algoritmo procura apenas minimizar o número absoluto de ligações, n – 1. Como o algoritmo de Kruskal processa as arestas por ordem de peso e, neste caso, todas as ordenações são válidas, a árvore resultante é altamente ramificada e arbitrária em termos espaciais. Ligações longas têm exatamente a mesma prioridade que ligações curtas, o que gera caminhos logicamente válidos, mas geometricamente maus, desconectando estações vizinhas que estão à face da rua.

- **MST com custo por distância:** Ao introduzir a distância em quilómetros, o algoritmo ordena e prioriza sistematicamente as conexões geograficamente mais curtas. O subgrafo resultante organiza-se em *clusters* regionais densos (as estações centrais de Londres, onde as distâncias interestações são mais curtas, ligam-se rapidamente entre si, enquanto as linhas periféricas e extensas funcionam como ramais terminais únicos, cortando qualquer circuito redundante dispendioso.

A análise demostra que a função de custo define o propósito da otimização do grafo:

- Uma função baseada em custo uniforme foca-se na otimização abstrata da conectividade, sendo útil para simplificar redes lógicas ou protocolos de comunicação digitais onde a distância física não dita o custo de transmissão.

- Uma função baseada em distância geográfica traduz restrições do mundo real e de engenharia. O seu impacto visual e prático reflete um planeamento económico de infraestruturas (minimiza a quantidade total de carris/túneis a construir, mantendo toda a cidade conectada.

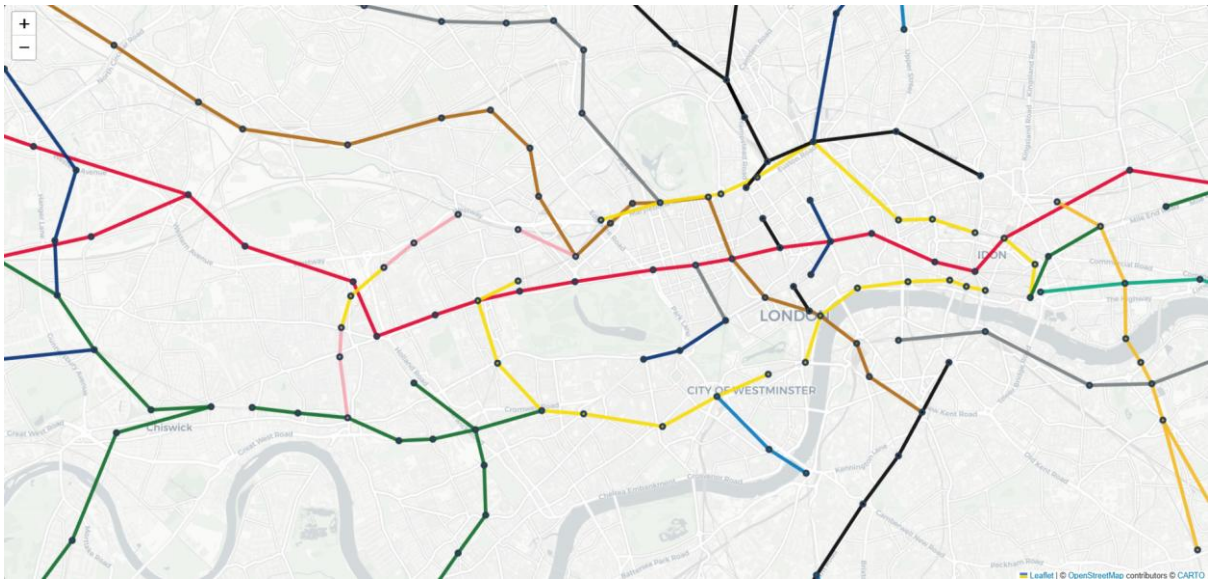


Imagem 12 – MST com custo uniforme

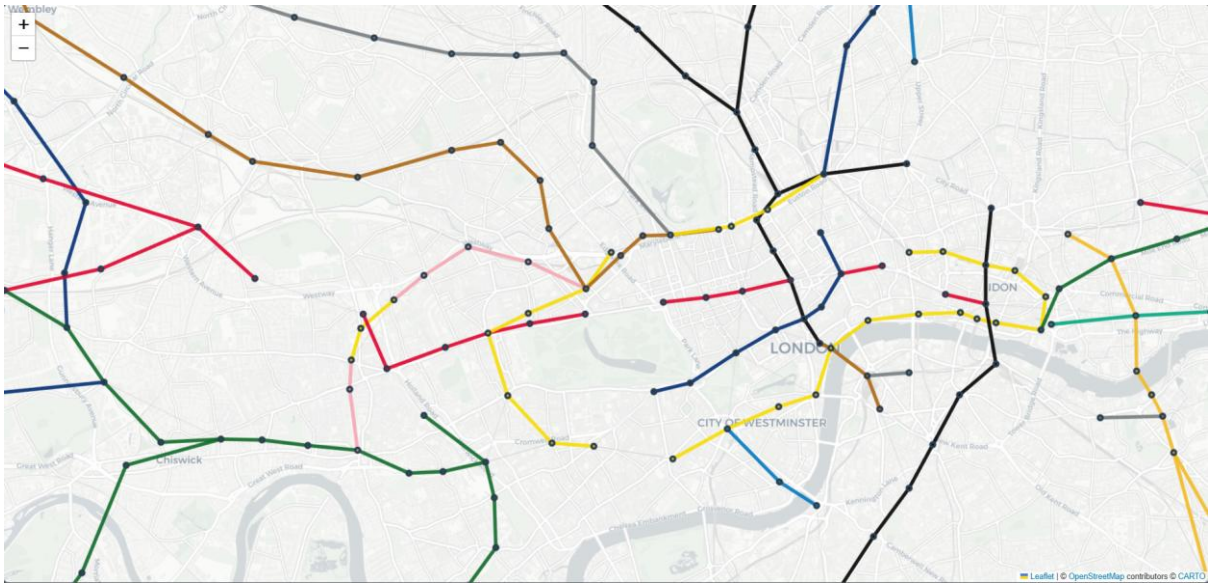


Imagem 13 – MST com custo por distância

Análise dos resultados, Discussão e Conclusões

Subseção 5.1: Questões e análises

- **Pergunta 1: Como a função de custo influencia o caminho obtido?**

  ➢ A função de custo define o critério de otimização que o algoritmo de procura, como o de Dijkstra, vai utilizar para determinar qual é o caminho mais curto.

  → **Custo uniforme:** Quando todas as arestas têm o mesmo peso = 1, o algoritmo foca-se exclusivamente na vertente do grafo, tendendo a minimizar o número total de estações percorridas.

  → **Distância geográfica:** Ao utilizar as coordenadas geográficas das estações, através da fórmula de Haversine, o algoritmo passa a minimizar a distância real, em quilómetros, entre a origem e o destino. O caminho resultante pode incluir mais estações se estas estiverem geograficamente mais próximas umas das outras.

- **Pergunta 2: Existe conflito entre menor distância e menor número de mudanças de linha?**

  ➢ Sim, existe um conflito entre estes dois fatores. O caminho que garante a menor distância física ou menor número de estações pode requerer que o passageiro mude várias vezes de linha para conseguir uma rota mais direta. Por outro lado, para garantir o menor número de mudanças de linha, ou até viajar sem transbordos, o passageiro poderá ter de se manter numa linha contínua que faz um percurso mais longo e orbital da rede, resultando numa distância percorrida significativamente maior do que se tivesse trocado de linha.

- **Pergunta 3: O caminho com menos estações é sempre o mais eficiente?**

  ➢ Não, o caminho com menos estações não é necessariamente o mais eficiente. A eficiência depende da métrica considerada:

→ **Distância geográfica variável:** No modelo do custo uniforme, todas as ligações contam como 1, mas na realidade a distância física entre estações vizinhas varia muito, as estações no centro de Londres são muito mais próximas entre si do que as estações nas extremidades.

→ **Custo do transbordo:** Um caminho com menos estações pode obrigar a múltiplos transbordos. Cada mudança de linha na vida real envolve o tempo de sair do comboio, caminhar pelos túneis de ligação e esperar pelo próximo comboio na plataforma. Por isso, um trajeto que passe por mais estações, mas que seja feito de forma direta, costuma de ser muito mais rápido, eficiente e confortável.

- **Pergunta 4: Qual o impacto das penalizações nos percursos?**

➢ As penalizações alteram o comportamento do algoritmo ao inflacionar artificialmente o peso de determinadas decisões.

→ **Penalização de mudanças de linha:** Ao adicionar um custo extra por cada transbordo, o algoritmo deixa de escolher caminhos excessivamente fragmentados. Ele passará a preferir rotas que mantenham o passageiro na mesma linha durante mais tempo, mesmo que isso implique percorrer mais estações, gerando soluções mais realistas e confortáveis.

→ **Penalização por congestionamento / Horas de ponta:** Ao sobrecarregar o peso de certas ligações em horários de pico, o impacto direto é que o algoritmo gera percursos alternativos que contornam as zonas críticas da rede. Isto permite desenhar simulações que dão prioridade à fluidez da viagem em detrimento da rota puramente mais curta.

- **Pergunta 5: Como diferentes funções de custo afetam o algoritmo de Dijkstra?**

➢ A função de custo determina o critério de otimização pelo algoritmo para determinar o caminho mínimo.

→ **Custo uniforme:** O algoritmo minimiza o número de estações percorridas, tratando todas as ligações como equivalentes.

→ **Distância geográfica:** O algoritmo minimiza a distância real em quilómetros, podendo resultar em caminhos com mais estações, mas geograficamente mais curtos.

→ **Hora de ponta:** O algoritmo evita ligações congestionadas em períodos de maior afluência, gerando percursos alternativos.

- **Pergunta 6: Porque o algoritmo de Kruskal não é adequado para encontrar caminhos mínimos?**
  - O algoritmo de Kruskal foi concebido para construir a Árvore Geradora Mínima, ligando todos os nós da rede com o menor custo total. Ao contrário do Dijkstra, não considera uma estação de origem nem de destino específicos, pelo que não garante o caminho mais curto entre dois pontos.

- **Pergunta 7: O que representa a árvore geradora mínima neste contexto?**

  - A MST representa a infraestrutura mínima necessária para manter todas as estações do metro de Londres conectadas sem redundância, eliminando ligações desnecessárias e mantendo a conectividade total da rede com o menor custo possível.

- **Pergunta 8: Como o modelo pode ser melhorado para representar situações reais?**

  - O modelo poderia ser melhorado incorporando dados reais como:
    - → Tempo de viagem entre estações;
    - → Frequência de comboios e tempo de espera;
    - → Tempo de transbordo em cada estação;
    - → Preço das tarifas por zona de viagem.

Subsecção 5.2 - Discussão Crítica

Ao longo do desenvolvimento deste trabalho, foram implementadas e analisadas diferentes abordagens para modelar e otimizar percursos na rede do metro de Londres. A utilização de grafos revelou-se uma solução eficaz para representar a rede, permitindo aplicar algoritmos de forma eficiente.

Em termos de vantagens, a implementação do algoritmo de Dijkstra em duas versões - própria e via NetworkX – permitiu validar os resultados comparar o desempenho de cada abordagem. A versão NetworkX mostrou-se mais rápida e simples de implementar, enquanto a versão própria ofereceu maior flexibilidade para incorporar novas funções de custo, como a penalização na hora de ponta. O algoritmo de Kruskal, por sua vez, permitiu identificar a infraestrutura mínima necessária para manter a rede conectada, o que pode ser útil em cenários de planeamento urbano.

Relativamente às limitações, o modelo desenvolvido não considera fatores relevantes da realidade como o tempo de espera nas estações, a frequência de comboios, o tempo de transbordo ou o preço das tarifas dependendo da zona de viagem. Estas limitações fazem com que os caminhos obtidos sejam teoricamente ótimos, mas nem sempre os mais práticos para um utilizador real.

No que diz respeito ao impacto das decisões de modelação, a escolha da função de custo revelou-se determinante nos resultados obtidos. O custo uniforme produz soluções simples, mas pouco realistas, enquanto a distância geográfica aproxima o modelo da realidade. A penalização na hora de ponta introduz um nível adicional de realidade, tornando o modelo mais dinâmico.

Em termos de complexidade computacional, o algoritmo de Dijkstra com fila de prioridade apresenta uma complexidade O (E log V), onde V representa o número de estações e E o número de ligações. O algoritmo de Kruskal tem uma complexidade de O (E log E), dominada pela ordenação das arestas.

Em conclusão, o trabalho demonstrou que a modelação de redes de transporte com grafos é uma abordagem boa e flexível, o algoritmo de Dijkstra revelou-se eficaz para encontrar caminhos mínimos entre dois pontos, sendo a escolha da função de custo determinante no resultado obtido. O algoritmo de Kruskal, embora não seja adequado para encontrar caminhos mínimos, é útil para identificar a infraestrutura mínima necessária para manter a rede conectada – algo com aplicações práticas no planeamento e otimização de redes de transporte urbano. A principal diferença entre os dois algoritmos reside no seu objetivo: enquanto o Dijkstra otimiza um percurso entre dois pontos, o Kruskal otimiza a estrutura global da rede. A qualidade das soluções obtidas por ambos depende da riqueza dos dados e das funções de custo utilizadas.

Declaração de utilização de ferramentas de Inteligência Artificial Generativa

- No âmbito do desenvolvimento do projeto ***LondonNetworkGraph***, declaramos ter utilizado Inteligência Artificial em tarefas de formatação visual. A intervenção da IA limitou-se apenas à estrutura e sintaxe do código HTML necessário para a renderização do mapa interativo. Concretamente, o seu uso incidiu sobre:

  → A criação da função ***build_legend***, que cria a legenda estática das linhas do metro.

  → A integração de elementos HTML na criação do mapa dentro do método ***visualize***.

Referências

- [1] "Dijkstra's Algorithm in Python," DataCamp. [Online]. Disponível em: https://www.datacamp.com/pt/tutorial/dijkstra-algorithm-in-python

- [2] "Kruskal's Algorithm in Python," PythonPool. [Online]. Disponível em: https://www.pythonpool.com/kruskals-algorithm-python/

- [3] "Folium Documentation," Folium. [Online]. Disponível em: https://folium.readthedocs.io/en/latest/

- [4] "Fast Haversine Approximation Python," Stack Overflow. [Online]. Disponível em: https://stackoverflow.com/questions/29545704/fast-haversine-approximation-python-pandas

- [5] OpenAI, "ChatGPT," versão GPT-4, 2025. [Online]. Disponível em: https://chat.openai.com

- [6] Google, "Gemini," 2025. [Online]. Disponível em: https://gemini.google.com

- [7] SmartDraw, "SmartDraw - Diagramming Software," 2025. [Online]. Disponível em: https://www.smartdraw.com/